

Banco de Preguntas y Respuestas Técnicas - Módulo Administrador (HTML, CSS y JavaScript)

Proyecto SGDM ASCEND (Sistema de Gestión Deportiva y Multideportiva)

Propósito del Documento:

Este documento contiene la batería exhaustiva de preguntas y respuestas técnicas enfocadas **exclusivamente en el Módulo Administrador**, abarcando la arquitectura HTML5, el sistema de diseño CSS3 (Flexbox, Grid, variables, unidades y animaciones) y la lógica de interacción en JavaScript del lado del cliente.

💡 *Nota pedagógica:* Cada acrónimo, unidad de medida (**fr**, **rem**, **vh**, etc.) o concepto técnico está formalmente definido y explicado dentro de cada respuesta para asegurar una defensa impecable frente a cualquier tribunal docente.

Índice Temático

1. [Bloque 1: Arquitectura Semántica y Estructura HTML5 del Admin \(Preguntas 1 a 7\)](#)
2. [Bloque 2: Sistema de Diseño, Unidades y Variables CSS3 \(Preguntas 8 a 16\)](#)
3. [Bloque 3: Maquetación Responsiva y Layout del Panel \(Preguntas 17 a 23\)](#)
4. [Bloque 4: Componentes de UI, Tablas y Animaciones CSS \(Preguntas 24 a 30\)](#)
5. [Bloque 5: Lógica JavaScript, Eventos y DOM en el Admin \(Preguntas 31 a 38\)](#)
6. [Bloque 6: Enrutamiento de Maquetas, Seguridad y Backend \(Preguntas 39 y 40\)](#)

Bloque 1: Arquitectura Semántica y Estructura HTML5 del Admin

1. ¿Cuál es la estructura HTML5 principal del panel de administración y qué etiquetas semánticas utiliza?

Respuesta:

La estructura del panel de administración sigue una jerarquía semántica estricta basada en el estándar **HTML5** (la quinta revisión del lenguaje de marcado estándar de la web), dividida en cuatro bloques principales:

<code><header class="cabecera-principal"></code> (Logo + Perfil/Logout)	
<code><aside/nav class="menu-lateral"></code> (Enlaces y acordeones)	<code><main id="area-contenido" class="area-contenido"></code> - <code><section class="encabezado-seccion"></code> - <code><section class="contenedor-tarjetas"></code> - <code><section class="tabla-responsiva"></code>
<code><footer class="pie-pagina"></code> (Copyright + Versión del SGDM)	

1. **<header>**: Contiene la identidad institucional (logo) y la barra superior de acciones rápidas (avatar y menú desplegable de perfil).
2. **<nav> / <aside> (clase .menu-lateral)**: Aloja la navegación vertical jerárquica con enlaces a los módulos (Usuarios, Torneos, Equipos, Auditoría, Reportes).
3. **<main> (id area-contenido)**: Representa el núcleo temático exclusivo de la página; es el área donde se inyectan las vistas de los CRUD (*Create, Read, Update, Delete*).
4. **<footer>**: Pie de página estructural con información legal y enlaces secundarios.

¿Por qué es importante? Evita el llamado "Div Soup" (abuso indiscriminado de etiquetas **<div>** sin significado) y permite a los motores de búsqueda y lectores de pantalla entender la jerarquía funcional de la aplicación.

2. ¿Por qué utilizaron las etiquetas **<details>** y **<summary>** en las tablas de datos del Administrador?

Respuesta:

Las etiquetas **<details>** y **<summary>** se utilizaron para implementar el **Menú Desplegable de Acciones en Tablas** (clase **.acciones-desplegables**) de forma **nativa en HTML5**:

- **<details>**: Es un elemento nativo que actúa como un contenedor revelador de contenido (*Disclosure Widget*). Por defecto está cerrado, y al recibir un clic se le agrega automáticamente el atributo booleano **open**.
- **<summary>**: Define el encabezado visible o etiqueta interactiva del botón (por ejemplo, el botón "Acciones ▾" con tres puntos).

Ventajas Técnicas:

1. **Cero Dependencia de JavaScript para la Apertura**: Funciona automáticamente en el navegador sin necesidad de programar escuchadores **addEventListener('click')** por cada fila de la tabla.
 2. **Accesibilidad Nativa (A11y)**: Permite que usuarios con lectores de pantalla o que navegan con el teclado (usando la tecla **Espacio** o **Enter**) puedan abrir y cerrar el menú sin requerir programación adicional de atributos ARIA.
 3. **Rendimiento Óptimo**: Al ser procesado directamente por el motor C++ del navegador, no consume ciclos de CPU en el hilo principal de JavaScript (*Main Thread*).
-

3. ¿Cómo están estructurados los formularios administrativos para garantizar accesibilidad y usabilidad?

Respuesta:

Los formularios del Administrador (clase **.formulario-estandar**) implementan buenas prácticas de diseño de formularios:

1. **<fieldset> y <legend>**: Agrupan visual y semánticamente campos relacionados (por ejemplo: **<fieldset><legend>Datos de Acceso</legend>...</fieldset>**), lo que facilita la comprensión en pantallas densas.
2. **Vinculación Explícita de <label> con <input>**: Cada etiqueta **<label>** utiliza el atributo **for="id_del_campo"**, el cual coincide de manera unívoca con el **id** del input. Esto permite que al

hacer clic sobre el texto de la etiqueta, el foco se sitúa inmediatamente en la caja de texto (aumentando el área de toque o *Hit Target*).

3. **Indicadores de Obligatoriedad y Tipado Correcto:** Se utilizan tipos de datos específicos de HTML5 como `type="email"`, `type="password"`, `type="number"` y la clase `.campo-requerido` para validación cruzada.
-

4. ¿Qué es WAI-ARIA y qué atributos de accesibilidad se contemplaron en la interfaz del Administrador?

Respuesta:

WAI-ARIA (*Web Accessibility Initiative - Accessible Rich Internet Applications*) es una especificación técnica de la W3C que añade semántica extra mediante atributos especiales para personas con discapacidades que utilizan tecnologías de asistencia (como lectores de pantalla NVDA o JAWS):

- `aria-label="Abrir menú de navegación"`: Proporciona una descripción textual a botones que solo contienen un ícono gráfico (como el botón hamburguesa móvil `.boton-menu-movil`).
 - `aria-expanded="true/false"`: Comunica dinámicamente al lector si un submenú desplegable o acordeón está actualmente abierto o cerrado.
 - `role="alert"` o `aria-live="polite"`: Utilizado en el contenedor de notificaciones flotantes (`.contenedor-notificaciones`) para que el lector de pantalla anuncie de inmediato mensajes críticos (por ejemplo: "Usuario eliminado exitosamente") sin interrumpir la navegación del usuario.
-

5. ¿Por qué las tablas utilizan estrictamente `<thead>`, `<tbody>`, `<th>` y `<td>`?

Respuesta:

Porque en un panel de administración las tablas son el componente central de visualización de datos:

- `<thead>` y `<th>` (**Table Header**): Definen la fila de encabezados. El navegador los trata como títulos de columna, aplicando estilos de mayor jerarquía (`font-weight: 600; text-transform: uppercase;`).
 - `<tbody>` y `<td>` (**Table Data**): Contienen los registros de datos dinámicos.
 - **Ventaja para el diseño responsivo:** Esta separación permite que el CSS aplique reglas como `.tabla-estandar tbody tr:hover` para iluminar la fila activa y facilita que en dispositivos móviles se pueda configurar `overflow-x: auto;` manteniendo la integridad del encabezado.
-

6. ¿Qué función cumple el elemento `<meta name="viewport" content="width=device-width, initial-scale=1.0">` en la cabecera?

Respuesta:

Es la directiva obligatoria para habilitar el **Diseño Web Responsivo** (*Responsive Web Design - RWD*):

- `width=device-width`: Le indica al navegador que el ancho del área de visualización (*Viewport*) debe coincidir con el ancho físico real de la pantalla del dispositivo en píxeles CSS (evitando que el celular asuma un ancho de escritorio de 980px y muestre el sitio en miniatura con zoom alejado).
 - `initial-scale=1.0`: Establece el nivel de zoom inicial al 100% al cargar la página por primera vez.
-

7. ¿Por qué se utilizó `box-sizing: border-box;` de forma global en los estilos del Administrador?

Respuesta:

El modelo de caja tradicional de CSS (*Content-Box*) calcula el ancho total de un elemento sumando:
$$\text{\texttt{\text{Ancho Total}}} = \text{\texttt{\text{width}}} + \text{\texttt{\text{padding}}} + \text{\texttt{\text{border}}}$$

Si a un input de un formulario le asignas `width: 100%;` y un `padding: 10px;`, el input desbordará y sobresaldrá por fuera de su contenedor.

Al aplicar `box-sizing: border-box;`, el navegador calcula el ancho de manera intuitiva:
$$\text{\texttt{\text{Ancho Total}}} = \text{\texttt{\text{width}}} \text{ (el padding y el borde se absorben hacia adentro)}$$
 Esto garantiza que los campos de formularios y paneles al 100% nunca rompan la grilla ni generen scroll horizontal no deseado.

Bloque 2: Sistema de Diseño, Unidades y Variables CSS3

8. ¿Qué son las Variables CSS (*Custom Properties*) y por qué se definen dentro del selector `:root`?

Respuesta:

Las **Variables CSS** son entidades definidas por el autor de la hoja de estilos que contienen valores específicos reutilizables a lo largo de todo el documento. Se declaran con dos guiones iniciales (ej. `--color-primario: #4b6584;`) y se consumen mediante la función `var(--nombre)`.

¿Por qué se declaran en `:root`?

- `:root` es una pseudo-clase de CSS que hace referencia al elemento raíz del documento HTML (`<html>`).
- Al declararlas en `:root`, las variables adquieren un **Ámbito Global (*Global Scope*)**, lo que significa que están disponibles en cascada para todos los componentes, vistas y módulos del sistema de administración.

9. ¿Qué variables de diseño (*Design Tokens*) están configuradas para el Administrador en `base.css` y qué propósito tienen?

Respuesta:

En `codigo_fuente/publico/css/admin/base.css` se configuró una paleta de colores sobria y ergonómica diseñada para software administrativo:

Variable	Valor	Significado y Propósito en el Módulo Admin
<code>--color-fondo-principal</code>	<code>#f8f9fa</code>	Gris muy claro neutro que reduce el cansancio ocular en sesiones largas.
<code>--color-fondo-panel</code>	<code>#ffffff</code>	Blanco puro para tarjetas, tablas y modales que genera contraste y orden.
<code>--color-texto-principal</code>	<code>#343a40</code>	Gris grafito oscuro (evita el negro puro <code>#000000</code> para mejorar la legibilidad).

Variable	Valor	Significado y Propósito en el Módulo Admin
--color-texto-secundario	#6c757d	Gris medio para subtítulos, etiquetas y marcas de tiempo (<i>timestamps</i>).
--color-primario	#4b6584	Azul pizarra institucional para botones de acción y enlaces activos.
--color-exito	#45b676	Verde esmeralda suave para estados activos y confirmaciones exitosas.
--color-peligro	#e05d5d	Rojo suave para eliminaciones, bloqueos de usuarios y errores.
--sombra-suave	0 1px 3px rgba(0,0,0,.06)	Sombra sutil de elevación (elevación z-index visual) para tarjetas y tablas.
--radio-borde	6px	Curvatura sutil en esquinas de botones y paneles que aporta estética moderna.

10. ¿Qué significa la unidad **rem** y por qué es superior a los píxeles (**px**) para tipografía y espaciados?

Respuesta:

- **Definición de rem (Root em):** Es una **unidad de medida relativa** que se calcula en función del tamaño de fuente (*font-size*) del elemento raíz (`<html>`).
- **Cálculo Base:** Por defecto en todos los navegadores web, **1rem = 16px**. Por lo tanto:
 - **0.8rem = 12.8px** (encabezados de tablas y badges pequeños).
 - **1.5rem = 24px** (subtítulos de sección).
 - **2.2rem = 35.2px** (números de métricas en tarjetas de estadísticas).

¿Por qué es superior a px?

1. **Accesibilidad Universal:** Si un usuario con problemas de visión configura en su navegador un tamaño de texto predeterminado más grande (ej. 20px), todos los textos y espaciados basados en **rem** escalan proporcionalmente de forma automática. Si usáramos píxeles fijos (**px**), la interfaz ignoraría la preferencia del usuario.
2. **Escalabilidad Global:** Si se desea redimensionar toda la interfaz para pantallas gigantes (4K), basta con ajustar una sola línea: `html { font-size: 18px; }`.

11. ¿Qué significan las unidades **vh** y **vw** y cómo se usan en el Administrador?

Respuesta:

Son **unidades del Viewport (Área de Visualización Visible)**:

- **1vh (Viewport Height):** Equivale al **1% de la altura total de la ventana** del navegador. **100vh** representa el 100% exacto de la altura de la pantalla del usuario.
- **1vw (Viewport Width):** Equivale al **1% del ancho total de la ventana** del navegador. **100vw** representa el 100% exacto del ancho.

Uso en el Administrador: En el `body.tema-admin` se define:

```
body.tema-admin {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
```

Esto le garantiza al sistema que el contenedor del administrador ocupe **al menos toda la altura de la pantalla**, permitiendo que el footer se sitúe siempre abajo aunque la tabla tenga pocos registros.

12. ¿Qué significa la unidad `fr` en CSS Grid y dónde se utiliza en el Administrador?

Respuesta:

- **Definición de `fr` (*Fractional Unit* - Unidad Fraccionaria):** Es una unidad de medida exclusiva de **CSS Grid** que representa una fracción del espacio libre disponible dentro del contenedor de la cuadrícula.
- **Cómo funciona el cálculo:** Si defines `grid-template-columns: 1fr 1fr;`, el navegador divide el ancho disponible en $1 + 1 = 2$ partes iguales y le asigna el 50% del espacio a cada columna, descontando automáticamente el valor del `gap`.

Uso en el Administrador (`componentes.css`):

```
.cuadrícula-permisos {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 15px;
}
```

Se utiliza en la pantalla de roles y permisos para dividir la lista de casillas de verificación (*checkboxes*) en dos columnas simétricas y perfectamente alineadas.

13. ¿Qué diferencia hay entre `rem` y `em`?

Respuesta:

Unidad	Elemento de Referencia	Comportamiento en Anidamiento
<code>rem</code> (<i>Root em</i>)	Toma como referencia siempre el elemento raíz (<code><html></code>).	Predecible: <code>1.5rem</code> siempre medirá lo mismo en cualquier lugar del documento.
<code>em</code>	Toma como referencia el <code>font-size</code> de su elemento padre directo .	Efecto Cascada Multiplicativo: Si anidas un <code>em</code> dentro de otro <code>em</code> , el tamaño se multiplica acumulativamente (pudiendo generar tamaños gigantes o minúsculos por error).

Regla aplicada en el proyecto: Se utiliza **rem** para tipografías y espaciados estructurales por su estabilidad y consistencia.

14. ¿Qué es **mix-blend-mode: multiply;** en el contenedor del logo del header?

Respuesta:

mix-blend-mode es una propiedad de CSS3 que define cómo debe mezclarse el color o los píxeles de un elemento con los colores del fondo sobre el que está colocado (similar a los modos de fusión de capas en Adobe Photoshop):

- El valor **multiply (Multiplicar)** multiplica los valores de los canales de color del logo con los del fondo.
 - **Efecto práctico:** Si la imagen del logo tiene un fondo blanco sólido original, el modo **multiply** vuelve el blanco completamente invisible/transparente, integrando el logo sobre la barra superior sin necesidad de editar la imagen en un software externo.
-

15. ¿Qué significa **transform-origin** en las animaciones de los menús desplegados?

Respuesta:

transform-origin establece el **punto de anclaje o pivote** a partir del cual se aplican las transformaciones geométricas de CSS (**scale()**, **rotate()**, etc.):

- Por defecto, el punto de origen es el centro (**50% 50%**).
- Al definir en los menús desplegados:

```
details[open].acciones-desplegables .acciones-lista {  
  transform-origin: top right;  
}
```

La animación de escala (**scale(0.90)** a **scale(1)**) se despliega emergiendo **desde la esquina superior derecha** (donde el usuario hizo clic en el botón), logrando una sensación visual natural y fluida.

16. ¿Qué función cumple la curva **cubic-bezier(0.2, 0.8, 0.2, 1)** en las transiciones del Admin?

Respuesta:

Una función **cubic-bezier** define una curva de aceleración personalizada (*Timing Function*) para transiciones y animaciones CSS, regulando la velocidad con la que ocurre el cambio a lo largo del tiempo:

- A diferencia de una animación lineal aburrida (**linear**), la curva **cubic-bezier(0.2, 0.8, 0.2, 1)** implementa un efecto de **desaceleración suave (Ease-Out)**: arranca rápidamente y frena de manera amortiguada y elegante al final, imitando la física del mundo real.
-

Bloque 3: Maquetación Responsiva y Layout del Panel

17. ¿Por qué el panel de administración fue diseñado con enfoque Mobile First?

Respuesta:

Mobile First es una estrategia de ingeniería web donde los estilos base se escriben pensando en las limitaciones de una pantalla pequeña (móviles de 360px), y luego se expanden hacia pantallas grandes mediante `@media (min-width: ...)`:

1. **Rendimiento:** Los dispositivos móviles procesan el CSS base más liviano sin tener que sobrescribir reglas pesadas de escritorio.
2. **Priorización de la Información:** Obliga a estructurar primero lo verdaderamente importante (tablas esenciales, acciones clave) antes de distribuir paneles en múltiples columnas.
3. **Mantenimiento:** El código crece de forma modular y progresiva (*Progressive Enhancement*).

18. ¿Cómo cambia el layout del Administrador entre la versión móvil y la de escritorio?

Respuesta:

El cambio se gestiona mediante una única propiedad clave en `.cuerpo-admin`:

```
/* 1. MÓVIL (Por defecto fuera de Media Query) */
.cuerpo-admin {
  display: flex;
  flex-direction: column; /* Apilado vertical: menú arriba/oculto, contenido abajo */
}

/* 2. ESCRITORIO (A partir de 768px) */
@media (min-width: 768px) {
  .cuerpo-admin {
    flex-direction: row; /* En paralelo: Menú lateral a la izquierda + Área elástica a la derecha */
  }
}
```

- **En móviles:** El menú lateral `.menu-lateral` ocupa el 100% del ancho y está oculto (`display: none;`) hasta que el usuario toca el botón hamburguesa (`.boton-menu-movil`).
- **En escritorio:** La barra hamburguesa desaparece (`display: none;`), el menú lateral queda visible fijo con ancho de `260px` y el área de contenido (`.area-contenido`) se estira elásticamente con `flex: 1;`.

19. ¿Por qué se utilizó Flexbox para el layout principal del Admin en lugar de CSS Grid?

Respuesta:

Por la naturaleza **unidimensional (1D) y elástica** del panel administrativo:

1. **Contenido Elástico con `flex: 1`:** La barra lateral tiene un ancho predecible y el área de trabajo central debe absorber de forma fluida el 100% del espacio restante, adaptándose automáticamente a pantallas de 1366px, 1920px o monitores Ultrawide.

2. **Facilidad de Colapso:** Ocultar la barra lateral con JavaScript o colapsarla a formato íconos no requiere recalcular matrices de grillas complejas.
 3. **Sticky Footer Nativo:** Flexbox resuelve el pie de página fijo al fondo con solo dos reglas (`min-height: 100vh` en el body y `margin-top: auto` en el footer).
-

20. ¿Qué significa `flex: 1;` en el contenedor `.area-contenido`?

Respuesta:

`flex: 1;` es una propiedad abreviada (*shorthand*) que configura tres valores de Flexbox simultáneamente:

1. **`flex-grow: 1;`** (Factor de Crecimiento): Le indica al contenedor que tiene permiso para **crecer y absorber todo el espacio sobrante** disponible en el eje principal.
 2. **`flex-shrink: 1;`** (Factor de Reducción): Permite que el elemento se encoja proporcionalmente si el espacio total disminuye para evitar desbordes.
 3. **`flex-basis: 0%;`** (Base Inicial): Inicia el cálculo del tamaño desde cero, distribuyendo el espacio de forma equitativa y fluida.
-

21. ¿Cómo se logró que las tablas del Administrador sean 100% responsivas sin romper la pantalla en celulares?

Respuesta:

En `componentes.css` se aplicó la técnica de **Contenedor Desplazable Autónomo**:

```
.tabla-estandar {  
  display: block;  
  width: 100%;  
  overflow-x: auto;  
  white-space: nowrap;  
}
```

- **`display: block;`**: Transforma la tabla para que se comporte como un bloque contenedor estándar.
 - **`overflow-x: auto;`**: Si el ancho total de las columnas de la tabla supera el ancho físico de la pantalla del celular, el navegador habilita una **barra de desplazamiento horizontal exclusiva dentro de la tabla**.
 - **`white-space: nowrap;`**: Evita que los textos largos (como correos electrónicos o nombres de torneos) se quiebren en múltiples renglones feos, manteniendo la fila uniforme.
 - **Resultado:** La interfaz general de la página nunca sufre desborde (*overflow lateral*), manteniendo el header y el sidebar intactos mientras la tabla puede deslizarse con el dedo cómodamente.
-

22. ¿Qué función cumple la propiedad `overflow-x: hidden;` en `.area-contenido`?

Respuesta:

Actúa como un **cortafuegos o barrera de seguridad visual**:

- Si algún contenido interno dinámico (un texto largo sin espacios, un iframe o un componente mal dimensionado) intenta ensancharse más allá de la pantalla, `overflow-x: hidden;` recorta el excedente invisiblemente.
- Esto previene el clásico y molesto error de interfaz donde toda la página web "baila" o se mueve lateralmente en teléfonos móviles.

23. ¿Por qué el Administrador tiene clases dinámicas `.en-inicio` y `.en-interna` en el `<body>`?

Respuesta:

Para implementar el patrón de **Cabecera Contextual Adaptable**:

- **En el Dashboard (`.en-inicio`)**: La cabecera muestra el resumen del perfil del administrador completo con su avatar y bienvenida.
- **En pantallas internas de trabajo (`.en-interna`)** (como formularios de edición de usuarios o tablas de auditoría): El header simplifica su diseño (`.en-interna .perfil-admin { display: none; }`) para maximizar el área vertical útil de trabajo y evitar distracciones visuales.

Bloque 4: Componentes de UI, Tablas y Animaciones CSS

24. ¿Cómo están estructuradas las Tarjetas de Métricas Estadísticas del Dashboard (`.tarjeta-estadistica`)?

Respuesta:

Las tarjetas del dashboard siguen un patrón de diseño limpio y minimalista:

```
.tarjeta-estadistica {
  background: var(--color-fondo-panel);
  padding: 24px;
  border-radius: var(--radio-borde);
  flex: 1;
  text-align: center;
  box-shadow: var(--sombra-suave);
  border: 1px solid var(--color-borde);
  transition: transform 0.2s, box-shadow 0.2s;
}

.tarjeta-estadistica:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 6px rgba(0,0,0,0.08);
}
```

- **Efecto Micro-interactivo :hover**: Al pasar el cursor del ratón, la tarjeta se eleva 2 píxeles (`transform: translateY(-2px)`) y proyecta una sombra ligeramente más profunda, proporcionando retroalimentación visual táctil (*Affordance*).
- **Jerarquía de Información**: El título del indicador está arriba en mayúsculas discretas (`font-size: 0.9rem`), y el número de la métrica (ej. "1,420") se destaca en tamaño gigante (`font-size: 2.2rem`;

```
color: var(--color-primario));
```

25. ¿Qué es y cómo funciona el sistema de Notificaciones Flotantes (*Toast Notifications*)?

Respuesta:

Un **Toast** es una notificación visual no intrusiva que aparece sobre la interfaz para informar el éxito o fracaso de una acción (por ejemplo: "Usuario actualizado correctamente"):

1. Contenedor Fijo:

```
.contenedor-notificaciones {  
  position: fixed;  
  bottom: 20px;  
  right: 20px;  
  z-index: 9999;  
  pointer-events: none; /* No bloquea los clics en la página debajo */  
}
```

2. Animación de Entrada y Salida con @keyframes:

- Al crearse, la notificación sube suavemente desde el fondo (`deslizarArriba`: de `translateY(20px)` a `translateY(0)` y `opacity: 1`).
- A los 3 segundos, JavaScript le añade la clase `.ocultar`, activando la animación `desvanecer` (`opacity: 0`; `translateY(-10px)`), tras lo cual el nodo es eliminado del DOM.

26. ¿Qué significa `pointer-events: none;` y `pointer-events: auto;` en las notificaciones toast?

Respuesta:

- **`pointer-events: none;`** en el contenedor padre: Hace que el contenedor de notificaciones sea **"invisible" para los clics del ratón**. Si una esquina del contenedor cubre un botón de la tabla, el usuario puede seguir haciendo clic en el botón sin que el contenedor transparente lo bloquee.
- **`pointer-events: auto;`** en el toast individual: Restaura la capacidad de recibir clics exclusivamente sobre el cartel verde de la notificación para que el usuario pueda cerrarlo manualmente si lo desea.

27. ¿Qué es `z-index` y cómo se gestionan los niveles de apilamiento en el Administrador?

Respuesta:

`z-index` es la propiedad de CSS que controla el orden de apilamiento en el eje tridimensional Z (profundidad hacia el usuario) para elementos posicionados (`relative`, `absolute`, `fixed` o `sticky`):

Capa Z-Index en el Administrador:

<code>z-index: 9999</code>	→	Notificaciones Toast (<code>.contenedor-notific.</code>)
<code>z-index: 100</code>	→	Modales y Menús de Perfil (<code>.dropdown-content</code>)

z-index: 10	→ Cabecera Fija (.cabecera-principal)
z-index: 1	→ Tarjetas y Tablas (Flujo normal)

Esta escala estandarizada previene que elementos como las tablas o botones tapen accidentalmente los menús desplegables.

28. ¿Por qué se utilizan pseudo-elementos como `::before` y `::after` en la interfaz?

Respuesta:

Los **pseudo-elementos** permiten insertar contenido cosmético o decorativo en el árbol de renderizado sin ensuciar el marcado HTML:

- Se utilizan para generar pequeños triángulos indicadores en menús desplegables (*tooltips*), barras decorativas de acento a la izquierda de enlaces activos (`border-left: 3px solid var(--color-primario);`) o íconos gráficos de estado.

29. ¿Cómo se logra la personalización de los botones de acción (`.boton-peligro`, `.boton-exito`, `.boton-secundario`)?

Respuesta:

Se utilizó el patrón de clases utilitarias semánticas de **Arquitectura de Botones**:

- **Base compartida:** Todos comparten la misma tipografía, padding equilibrado (`6px 12px`), radio de borde (`var(--radio-borde)`) y transición de suavizado (`transition: all 0.2s ease`).
- **Modificadores por intención de uso:**
 - `.boton-accion`: Fondo azul primario para acciones estándar (Guardar, Filtrar).
 - `.boton-exito`: Fondo verde para aprobaciones o reactivaciones de cuentas.
 - `.boton-peligro`: Contorno o fondo rojo para eliminaciones destructivas.
 - `.boton-secundario`: Fondo neutro y borde gris para botones de "Cancelar" o "Volver".

30. ¿Qué ventaja ofrece el estado `:focus-visible` frente al `:focus` tradicional?

Respuesta:

- `:focus` activa el contorno del elemento siempre que recibe el foco, incluso cuando un usuario hace clic con el ratón (lo que a veces genera bordes azules poco estéticos que confunden al usuario común).
- `:focus-visible` es una pseudo-clase moderna de accesibilidad inteligente: **solo dibuja el anillo de enfoque si el usuario está interactuando mediante el teclado** (tecla `Tab`), respetando la estética para usuarios de ratón pero garantizando accesibilidad total para personas que navegan con teclado.

Bloque 5: Lógica JavaScript, Eventos y DOM en el Admin

31. ¿Por qué todo el código JavaScript de `admin.js` está envuelto en `document.addEventListener('DOMContentLoaded', ...)`?

Respuesta:

El evento `DOMContentLoaded` se dispara cuando el navegador ha terminado de descargar y parsear completamente el documento HTML y ha construido la estructura del árbol **DOM (Document Object Model)**:

- Si intentáramos ejecutar `document.getElementById('btnMenuMovil')` antes de este evento (por ejemplo, con un `<script>` en el `<head>`), el elemento aún no existiría en memoria y la función retornaría `null`.
 - Al esperar a `DOMContentLoaded`, nos aseguramos al 100% de que todos los botones, tablas y formularios están disponibles para asociarles escuchadores de eventos sin lanzar errores de ejecución.
-

32. ¿Cómo funciona la apertura y cierre del menú hamburguesa móvil en `admin.js`?

Respuesta:

Se gestiona mediante manipulación de clases CSS y captura de eventos:

```
const btnMenuMovil = document.getElementById('btnMenuMovil');
const menuLateral = document.querySelector('.menu-lateral');

if (btnMenuMovil && menuLateral) {
  btnMenuMovil.addEventListener('click', function(evento) {
    evento.stopPropagation();
    menuLateral.classList.toggle('activo');
  });
}
```

- `classList.toggle('activo')`: Es un método inteligente de JavaScript. Si la clase `'activo'` no está presente en el elemento, la agrega (abriendo el menú); si ya está presente, la remueve (cerrándolo).
 - En el CSS, `.menu-lateral.activo { display: block; }` se encarga de hacerlo visible en pantalla.
-

33. ¿Qué es la propagación de eventos (*Event Bubbling*) y por qué se usó `evento.stopPropagation()`?

Respuesta:

- **Event Bubbling (Burbujeo de Eventos)**: Es el mecanismo natural del navegador por el cual, cuando ocurre un evento en un elemento hijo (ej. clic en el botón hamburguesa), el evento "burbujea" hacia arriba a través de todos sus padres: `button` → `header` → `body` → `html` → `document`.
 - **El Problema**: En el panel tenemos un listener en el `document` que dice: "Si el usuario hace clic en cualquier parte de la pantalla, cierra el menú lateral".
 - **La Solución con `evento.stopPropagation()`**: Al hacer clic en el botón de abrir, detenemos la propagación para que el evento no llegue al `document`. Si no usáramos `stopPropagation()`, el botón abriría el menú e instantáneamente el listener del `document` lo cerraría en el mismo milisegundo.
-

34. ¿Cómo funciona el patrón de "Cerrar al hacer clic afuera" (*Click Outside*) implementado en el menú de perfil y sidebar?

Respuesta:

Se implementa evaluando el árbol de nodos con el método nativo `Node.contains()`:

```
document.addEventListener('click', function(evento) {  
    // Si el menú está abierto y el clic NO ocurrió dentro del menú ni dentro del  
    botón  
    if (menuLateral.classList.contains('activo') &&  
        !menuLateral.contains(evento.target) &&  
        !btnMenuMovil.contains(evento.target)) {  
        menuLateral.classList.remove('activo');  
    }  
});
```

- `evento.target` representa el elemento exacto sobre el cual el usuario hizo clic.
- Si `menuLateral.contains(evento.target)` es `false`, significa con total certeza matemática que el clic fue afuera del menú, procediendo a cerrarlo de forma limpia y transparente para el usuario.

35. ¿Cómo funciona la lógica de Menú Acordeón en el Sidebar del Administrador?

Respuesta:

El menú lateral implementa un **Acordeón Exclusivo (Solo un submenú abierto a la vez)**:

1. Con `querySelectorAll('.menu-enlace')` se seleccionan todos los ítems de navegación.
2. Al hacer clic en un enlace, JavaScript busca a su hermano adyacente inmediato con la propiedad `enlaceActual.nextElementSibling`.
3. Si el hermano es un submenú (`.menu-sublista`):
 - Cancela la navegación predeterminada con `evento.preventDefault()`.
 - Recorre todos los demás submenús abiertos con un bucle `.forEach()` y los cierra (`submenu.style.display = 'none'`).
 - Abre únicamente el submenú seleccionado (`posibleSubmenu.style.display = 'block'`).

36. ¿Cómo se implementó la confirmación de seguridad para acciones destructivas en el panel?

Respuesta:

Para evitar que un administrador borre un torneo o dé de baja a un usuario por un clic accidental:

```
const botonesDePeligro = document.querySelectorAll('.boton-peligro');  
  
botonesDePeligro.forEach(function(boton) {  
    boton.addEventListener('click', function(evento) {  
        const confirmado = window.confirm("¿Estás seguro de que deseas realizar  
esta acción? Es irreversible.");
```

```
        if (confirmado === false) {  
            evento.preventDefault(); // Cancela la navegación o el envío del  
formulario  
        }  
    });  
});
```

- Si el usuario presiona "Cancelar", `evento.preventDefault()` bloquea la acción original (no se envía la petición de eliminación al servidor).

37. ¿Cómo opera la validación personalizada en tiempo real de los formularios?

Respuesta:

En el evento `'submit'` del formulario:

1. Se seleccionan todos los inputs que poseen la clase `.campo-requerido`.
2. Se evalúa su contenido eliminando espacios en blanco con `campo.value.trim()` (para evitar que el usuario ingrese solo barras de espacio vacías).
3. Si el campo está vacío:
 - Se marca una bandera booleana `formularioValido = false`;
 - Se pinta dinámicamente un borde rojo de alerta en el input: `campo.style.border = "2px solid #e74c3c"`;
4. Si `formularioValido === false`, se ejecuta `evento.preventDefault()` para impedir que el formulario se envíe con datos incompletos.

38. ¿Qué es el método `trim()` en cadenas de texto y por qué es vital en seguridad y validación?

Respuesta:

El método `String.prototype.trim()` elimina todos los espacios en blanco (espacios simples, tabulaciones `\t` y saltos de línea `\n`) de ambos extremos de una cadena de texto sin modificar el texto del medio:

- *Ejemplo:* `"juan@correo.com ".trim()` → resulta en `"juan@correo.com"`.
- **Importancia:** Previene que un usuario envíe campos obligatorios llenos de espacios vacíos engañando a validaciones superficiales como `campo.value !== ""`, y limpia correos y nombres antes de enviarlos a la base de datos.

Bloque 6: Enrutamiento de Maquetas, Seguridad y Backend

39. ¿Qué función cumple el archivo `mockup-router.js` en el Administrador?

Respuesta:

`mockup-router.js` es un **Enrutador del Lado del Cliente basado en Hash (Client-Side Hash Router)**:

- **Objetivo:** Permite navegar e interactuar fluidamente entre todas las pantallas maquetadas del Administrador (Dashboard, Listado de Usuarios, Formulario de Edición, Auditoría) como si fuera una **SPA (Single Page Application)** antes de conectar el backend en PHP.

- **Mecanismo:**

1. Lee la ruta en el hash de la URL (`window.location.hash`, ej. `#usuarios/formulario?id=2`).
 2. Realiza una petición asíncrona con `fetch('usuarios/formulario.html')`.
 3. Inyecta el contenido recibido dentro de `<main id="area-contenido">`.
 4. Si recibe parámetros como `?id=2`, cambia automáticamente el título a *"Editar Usuario"*, modifica el botón a *"Actualizar Cambios"* y oculta el campo de contraseña.
-

40. ¿Cómo se conecta la interfaz del Administrador con el Backend PHP real y la seguridad OWASP?

Respuesta:

La interfaz del Administrador fue diseñada para integrarse limpiamente con la arquitectura PHP MVC del proyecto:

1. **Control de Acceso Basado en Roles (RBAC):** Al inicio de cada vista protegida en PHP se ejecuta `Sesion::requerirRol('administrador')`; si el usuario logueado es un jugador o un invitado, es redirigido inmediatamente al login.
2. **Protección contra Inyecciones SQL:** Todas las búsquedas y modificaciones de la interfaz utilizan el modelo `Usuario.php` con sentencias preparadas PDO (`$stmt->prepare("UPDATE usuarios SET ... WHERE id = :id")`).
3. **Trazabilidad y Auditoría (OWASP Top 10 - A09: Security Logging and Monitoring):** Cada vez que el administrador modifica un rol o borra un registro desde la interfaz, el sistema dispara automáticamente una inserción en la tabla `auditoria_cambios` guardando el ID del administrador, la acción ejecutada, la dirección IP y el estado anterior en formato JSON.